

APARTADOS DESARROLLADOS

Dentro de este documento se recopilan los apartados que se desarrollaron por el usuario I-004nf dentro del repositorio, especificando los documentos que lo conforman y cual es la funcionalidad de cada uno de ellos.

Máquinas IoT

El apartado de Máquinas IoT funciona para administrar y monitorear los dispositivos del local, permite ver el estado en el que se encuentran, identificar el uso de cada uno y el tiempo de uso que tiene cada uno de ellos con una barra de progreso.

Actualmente este apartado se encuentra detenido en su desarrollo debido a la falta de los dispositivos finales para realizar pruebas y verificar la funcionalidad completa de la misma.

Archivos Generados

maquinas.blade.php

Contiene la vista principal del módulo, incluyendo la estructura HTML, estilos y el código JavaScript encargado de simular el comportamiento de los dispositivos IoT.

- **Front-end:** Lo que el usuario visualiza es un conjunto de cards interactivas, permitiendo añadir, eliminar o editar la información estática que se carga nuevamente al recargar la página.
- **Back-end:** Compuesto de un script contiene las funciones para que el apartado funcione de manera local.

Funcionalidades

El módulo se compone de distintas constantes para la funcionalidad local, alguna de las más importantes son las siguientes:

- **machines:** Carga los registros de las lavadoras y secadoras preestablecidas para cargar en el sistema.
- **washBadge:** Coloca un color a la etiqueta de acuerdo al uso que le corresponda.
- **cfig:** Asigna un color al card de acuerdo al estado en el que se encuentre el equipo (Disponible, en uso o fuera de servicio).

Alguna de las funciones que contiene son:

- **addMachine():** Agrega de manera local los nuevos dispositivos y actualiza la vista para añadirlo después de registrarlo.
- **startCycle():** Inicia un ciclo de lavado predeterminado de 30 minutos y cambia el estado del dispositivo a En uso.
- **releaseCycle():** Reinicia las constantes del ciclo de lavado predeterminado, menciona el estado disponible para el dispositivo.

- **deleteMachine()**: Indica que dispositivos deben de ser eliminados del registro del sistema.
- **confirmDelete()**: Advertencia que indica que el registro seleccionado está a punto de eliminarse por completo del sistema.

Rutas

Para acceder al sistema requerimos de una ruta funcional para el menú lateral:

```
Route::get('/maquinas', function () { return view('pages.maquinas', ['title' => 'Máquinas IoT']); }->name('maquinas');
```

Corte de Caja

El Corte de Caja lleva el control de todos los movimientos monetarios que se generan cada día, estamos hablando de las ventas, retiros de dinero, gastos adicionales que el negocio puede tener y que debe llevar un registro de ellos.

Aun siendo un apartado que recopila los ingresos del negocio se conforma de un panel con distintas acciones que ayudan a facilitar el trabajo.

Archivos Generados

Controller

CajaController.php

Centra todos los modelos que llegamos a requerir para la funcionalidad, hacemos uso del paquete de DomPDF, este nos ayuda a crear el reporte que se genera cuando el usuario realiza un corte de caja, es perfecto para usarse en conjunto con Laravel. Algunas de las funciones más importantes que se manejan dentro del controlador son las siguientes:

- **corte()**: Realiza las operaciones y registra la información dentro de la base de datos.
- **generarCorte()**: Recopila los datos de cada corte caja para crear el reporte del día, también ayuda a registrar ciertos valores en otras tablas.
- **facturaGlobal**: Realiza el procedimiento completo recapitulando la información para ser enviada a Facturapi y generar la factura global de las ventas de cada día.

Models

Los models que fueron creados realizan funciones como puente entre la base de datos y el sistema, cada una de ellas tiene su propia tabla donde guarda la información y alguna de ellas tiene relaciones con otros models.

ConfiguraciónCaja.php

Este model se encarga únicamente de la edición que se encuentra en la opción de configuración, la información que recopila es usada dentro del reporte generado por cada corte de caja, no es una información que llegue a afectar los valores que se mandan en conjunto a Facturapi.

De forma predeterminada el sistema indica que al crear la tabla se realice un registro en automático con valores que ayudan a generar el reporte. Este mismo no contiene relación con las demás tablas.

CorteCaja.php

Este model toma la información necesaria para crear un registro en la tabla de cortes_caja, esta nos ayuda a tener un recuento acerca de los distintos cortes que se han realizado por parte de los empleados. Este contiene las siguientes relaciones con las demás tablas:

```
// Relaciones
public function user()
{
    return $this->belongsTo(User::class, 'user_id');
}

public function ventas()
{
    return $this->hasMany(Sale::class, 'corte_id');
}

public function movimientos()
{
    return $this->hasMany(MovimientoCaja::class, 'corte_id');
}

// Helpers para filtrar por tipo
public function gastos()
{
    return $this->hasMany(MovimientoCaja::class, 'corte_id')
        ->where('tipo', 'gasto');
}

public function retiros()
{
    return $this->hasMany(MovimientoCaja::class, 'corte_id')
        ->where('tipo', 'retiro');
}
```

MovimientoCaja.php

Este model se enfoca en guardar la información que se registra en la opción de Gastos Adicionales o Retiro Seguro en la tabla de movimientos_caja, dentro podemos encontrar las siguientes relaciones:

```
// Scope para filtrar por turno actual
public function scopeDelTurno($query)
{
    return $query->where('user_id', auth()->id())
        ->whereNull('corte_id');
}

public function scopeGastos($query)
{
    return $query->where('tipo', 'gasto');
}

public function scopeRetiros($query)
{
    return $query->where('tipo', 'retiro');
}

// Relación con el corte
public function corte()
{
    return $this->belongsTo(CorteCaja::class, 'corte_id');
}

// Relación con el usuario
public function user()
{
    return $this->belongsTo(User::class, 'user_id');
}
```

Migrations

El apartado de corte de caja se conforma de distintas tablas, gracias a estas podemos tener una mejor organización de la información que recopilamos en el sistema.

2026_06_04_041359_create_cortes_caja_table.php

La migración crea una nueva tabla dentro de la base de datos llamada corte_caja, esta recopila la información para generar un registro de los distintos cortes de caja que se realizan, esta tabla se conforma de los siguientes campos:

Campos	¿Qué recopila?
<i>user_id</i>	Toma el id del usuario que esta realizando el corte de caja.
<i>folio</i>	Asigna un folio generado por automático por el sistema.
<i>fecha_cierre</i>	Asigna el día en el que se esta realizando el registro.
<i>fondo_inicial</i>	Guarda el valor con el que se inició el turno.
<i>total_ingresos</i>	Guarda el valor total de todas las ventas que se generaron en el día.
<i>total_gastos</i>	Guarda el valor de los gastos que se registraron en el día.
<i>total_retiros</i>	Guarda el valor de los retiros que se registraron en el día.
<i>efectivo_esperado</i>	Guarda la cantidad que se espera tener en caja.
<i>efectivo_contado</i>	Guarda la cantidad real que el empleado tiene en caja.

<i>diferencia</i>	Menciona la diferencia que puede existir entre el efectivo esperado y el efectivo contado.
<i>facturado</i>	Registra si la factura está pendiente a realizarse o ya se ha realizado.
<i>facturapi_id</i>	Guarda el id que genera Facturapi.

2026_06_08_224048_create_configuracion_caja_table.php

La migración crea la tabla de configuracion_caja, esta guarda los cambios que se realizan en la información de los campos del reporte que se genera después del corte de caja. En esta encontramos los siguientes campos:

Campo	¿Qué recopila?
<i>id</i>	Valor que se le asigna al registro.
<i>fondo_inicial</i>	Valor monetario con el que se inicia la caja.
<i>nombre_negocio</i>	Guarda el nombre del negocio.
<i>direccion</i>	Guarda la dirección física del negocio.
<i>ciudad</i>	Guarda la ciudad donde se encuentra el negocio.
<i>telefono</i>	Guarda el número de teléfono del negocio.
<i>codigo_postal</i>	Guarda el código postal del negocio.

2026_06_09_005640_create_movimientos_caja_table.php

Esta migración crea la tabla de movimientos_caja, esta sirve para añadir los retiros y gastos que se generan en el día, esta tabla se conforma de los siguientes campos.

Campo	¿Qué recopila?
<i>id</i>	Valor que se le asigna al registro.
<i>corte_id</i>	Guarda el id que se relaciona con el corte de caja.
<i>tipo</i>	Campo que se define en automático para indicar si es registrado como un gasto o retiro.
<i>monto</i>	Guarda el valor monetario que se ingresa en el campo.
<i>concepto_o_responsable</i>	Guarda el concepto del gasto o la persona responsable a la que se le entrega el dinero.

<i>Fecha_turno</i>	Guarda la fecha en la que se realiza el movimiento.
--------------------	---

2026_06_10_222252_add_user_id_and_corte_id_to_sales_table.php

La migración tiene como objetivo el agregar nuevas columnas dentro de la tabla de sales, los campos que se agregan son los siguientes:

Campo	¿Qué recopila?
user_id	Guarda el id del usuario que realiza la venta.
corte_id	Guarda el id del corte de caja que lo engloba.

2026_06_10_222523_add_user_id_to_movimientos_caja_table.php

La migración agrega una nueva columna dentro de la tabla de movimientos_caja, teniendo la siguiente función:

Campo	¿Qué recopila?
user_id	Guarda el id del usuario que registra un gasto o retiro..

2026_06_10_223853_add_user_id_to_cortes_caja_table.php

La función de la migración agrega una nueva columna dentro de la tabla de cortes_caja, este campo es el siguiente:

Campo	¿Qué recopila?
user_id	Guarda el id del usuario que realiza el corte de caja.

2026_06_08_081206_add_reference_to_orders_table.php

esta última migración ingresa una nueva columna dentro de la tabla de orders, este nuevo campo es el siguiente:

Campo	¿Qué recopila?
reference	Guarda el campo de reference dentro de sales

Lógica JS

corte-caja.js

El corte-caja.js contiene parte de la lógica que interactúa con el archivo de caja.blade.php, la lógica que maneja es por parte del cliente sin entrar al servidor. Algunas funciones sirven para modificar valores que el usuario visualiza en la interfaz

- **actualizarReloj():** Permite que el reloj que aparece al lado del título principal se actualice en tiempo real.
- **actualizarInterfazCaja():** Recopila información de los gastos y retiros que se realizan durante el turno que se reflejan en los campos que el usuario visualiza.
- **abrirDetalle(corte):** Muestra los detalles acerca de los cortes de caja realizados anteriormente.

Algunas otras son funciones que interactúan con el controller para completar un proceso:

- **agregarGastoLocal():** Realiza una petición POST para enviar los datos del formulario a la tabla de movimientos_caja.
- **agregarRetiroLocal():** Realiza una petición POST para enviar los datos del formulario a la tabla de movimientos_caja.
- **calcularDenominaciones():** Calcula el valor monetario de acuerdo a los billetes registrados y su valor.
- **calcularDiferencia():** Calcula la diferencia que tiene la cantidad esperada con la cantidad contada de caja.
- **confirmarCierre():** Envía los datos recopilados al servidor, este mismo responde con un documento PDF que se descarga en automático (El documento que se genera es el reporte del día).
- **generarFacturaGlobal():** Envía todas las ventas al servidor de Facturapi, este mismo responde con la factura global del día en formato PDF y archivo XML que se genera en conjunto.

Aunque existan las funciones dentro del script es necesario incluir la funcionalidad de **Window** dentro del archivo, este funciona como un intermediario para comunicar lo que requiere Laravel de JavaScript, permitiendo agregar configuraciones dinámicas y rutas generadas por el servidor para que este disponibles globalmente, encontrando las siguientes dos dentro del código:.

- **window.cajaConfig:** Controla las variables que definimos de manera global, siendo las siguientes:
 1. **fondoInicial:** El dinero con el que abrió la caja.
 2. **ingresosEfectivo:** Total de ventas realizadas con pago en efectivo.
 3. **retirosAutorizados:** Retiros realizados durante el turno.
 4. **gastosOperativos:** Gastos registrados durante el turno.
 5. **totalBruto:** La sumatoria total de todas las ventas registradas durante el turno.
 6. **csrfToken:** Token de seguridad obligatorio de Laravel para validar las peticiones asíncronas.
 7. **fechaFormato:** Cadena de texto generada para nombrar el archivo PDF del reporte.

- **window.cajaRoutes:** Contiene las rutas encargadas de procesar tareas que se relacionan con el servidor (movimientos de caja, corte de caja, facturación global, etc.).

Blade

caja.blade.php

Esto contiene la interfaz gráfica del apartado de corte de caja, este se conforma de elementos que mencionan las métricas, ventas del turno, movimiento en caja, el recuento histórico de las ventas generadas, entre otras cosas más.

Componentes de la interfaz

Opciones Seleccionables

La pantalla se conforma de diferentes secciones:

1. **Opciones Seleccionables:** En esta se encuentran los botones para generar el corte de caja, genera la factura global y la configuración de los datos del reporte.
2. **Métricas del programa:** Recapitula datos de los ingresos del día, los egresos registrados y las ventas en efectivo y con tarjeta que se realicen.
3. **Balance del Turno:** Métricas que indican de manera más sencilla los ingresos y egresos del día, así como el total generado.
4. **Desglose por Servicio:** Muestra una lista de los productos vendidos, tanto la cantidad como el total de todas ellas.
5. **Caja - Efectivo:** Muestra valores acerca de los retiros y gastos que se realizan en el turno, teniendo los botones para ingresar los datos y registrarlos en el sistema.
6. **Historial de cortes:** Muestra un registro histórico de los cortes de caja realizados anteriormente.

Variables Globales

Al igual que el documento anterior, en la sección de las etiquetas de script se mencionan las variables globales y rutas con relación al servidor:

- **window.cajaConfig:** Controla las variables que definimos de manera global, siendo las siguientes:
 1. **fondolInicial:** El dinero con el que abrió la caja.
 2. **ingresosEfectivo:** Total de ventas realizadas con pago en efectivo.
 3. **retirosAutorizados:** Retiros realizados durante el turno.
 4. **gastosOperativos:** Gastos registrados durante el turno.
 5. **totalBruto:** La sumatoria total de todas las ventas registradas durante el turno.
 6. **csrfToken:** Token de seguridad obligatorio de Laravel para validar las peticiones asíncronas.
 7. **fechaFormato:** Cadena de texto generada para nombrar el archivo PDF del reporte.

- **window.cajaRoutes:** Contiene las rutas encargadas de procesar tareas que se relacionan con el servidor (movimientos de caja, corte de caja, facturación global, etc.).
 1. **movimiento:** Petición para registrar los movimientos realizados en el turno (Retiros Seguros o Gastos Adicionales).
 2. **generarCorte:** Realiza el cierre de turno, recibiendo todos los datos del formulario para realizar los cálculos necesarios y realizar un reporte del día en formato PDF.
 3. **facturaGlobal:** Comienza el proceso de facturación ante todas las ventas que no se encuentren declaradas ante el SAT, recibiendo como respuesta un formato PDF y un archivo XML relacionado a la factura realizada.
 4. **actualizarFondo:** Modifica los campos de la tabla de configuracion_caja.

Modals

En esta parte a lo que nos referimos a modal son las pantallas emergente que pueden generarse al realizar una acción en el sistema, existen varias con fines distintos para cada una de ellas.

- **modal-cierre:** Muestra un formato en donde se realiza el conteo del dinero en efectivo con el cual el usuario cierra la caja, se divide entre el valor de los billetes y la cantidad de moneda.
- **modal-gasto:** Abre un formulario de dos campos para registrar los gastos adicionales que se realicen en el turno.
- **modal-retiro:** Abre un formulario de dos campos para registrar los retiros autorizados durante el turno.
- **modal-confirmar-cierre:** Muestra un mensaje de advertencia que avisa que se está realizando el corte de caja.
- **modal-exito-cierre:** Muestra un mensaje de confirmación ante el cierre de caja que se realizó.
- **modal-factura-global:** Muestra el resumen de lo que el sistema va a facturar.
- **modal-detalle_corte:** Visualiza los detalles del corte de caja que se haya realizado.
- **modal-config:** Abre un formulario con campos editables para modificar los datos del reporte o el inicio de caja.

Llamado de Script

Al contener el script en otro archivo distinto es necesario mandarlo a llamar de alguna forma, teniendo en cuenta que es necesario el uso de las variables globales el llamado quedaría de la siguiente manera.

```

<script>
  window.cajaConfig = {
    fondoInicial: parseFloat("{{ $fondoInicial ?? 500 }}" ) || 0,
    ingresosEfectivo: parseFloat("{{ $ingresosEfectivo ?? 0 }}" ) || 0,
    retirosAutorizados: parseFloat("{{ $retirosAutorizados ?? 0 }}" ) || 0,
    gastosOperativos: parseFloat("{{ $gastosOperativos ?? 0 }}" ) || 0,
    totalBruto: parseFloat("{{ $totalBruto ?? 0 }}" ) || 0,
    csrfToken: "{{ csrf_token() }}",
    fechaFormato: "{{ now()->format('Y-m-d_H-i') }}"
  };

  window.cajaRoutes = {
    movimiento: "{{ route('caja.movimiento') }}",
    generarCorte: "{{ route('caja.generarCorte') }}",
    facturaGlobal: "{{ route('caja.facturaGlobal') }}",
    actualizarFondo: "{{ route('caja.actualizarFondo') }}"
  };
</script>

<script src="{{ asset('js/corte-caja.js') }}"></script>

```

En donde el primer script manda a llamar a las variables globales dentro de **window.cajaConfig** y las rutas son llamadas por **window.cajaRoutes** el segundo script manda a llamar el archivo como tal, mencionando la ruta.

Rutas

Para este apartado se especifican distintas distintas rutas que permiten ejecutar funciones necesarias:

```

Route::get('/caja', [CajaController::class, 'corte'])->name('caja');
Route::post('/caja/movimiento', [CajaController::class, 'movimiento'])->name('caja.movimiento');
Route::post('/caja/generar-corte', [CajaController::class, 'generarCorte'])->name('caja.generarCorte');
Route::post('/caja/factura-global', [CajaController::class, 'facturaGlobal'])->name('caja.facturaGlobal');
Route::put('/caja/configuracion/fondo', [CajaController::class, 'actualizarFondo'])->name('caja.actualizarFondo');

```

Calendario

El apartado del Calendario es un recurso visual para mostrar los próximos eventos registrados en el sistema, recupera la información de los pedidos y las suscripciones para mostrarla, aunque su funcionalidad se limita a solo mostrar los detalles del pedido o suscripción próxima a llegar.

Archivos Generados

Controller

CalendarController.php

Este archivo contiene la funcionalidad para consultar los registros de la base de datos, para su uso es necesario hacer uso del model de Order (Encargado de la información de los pedidos), y el de ClientSubscription (Encargado de la información de las suscripciones). Al no tener un uso más que un recurso adicional se conforma de una sola función:

- index(): Siendo la única función, se compone de varios elementos por dentro:
 1. Obtiene el mes y año.
 2. Crea una fecha base para cada mes.
 3. Calcula el mes anterior y el siguiente.
 4. Construye la información del calendario.
 5. Guarda la información de navegación y del día actual.
 6. Obtiene los pedidos y suscripciones.
 7. Genera la información usando peticiones AJAX.

Lógica JS

calendar-logic.js

En comparación con el controller, este contiene varias funciones necesarias para su funcionamiento, podemos verlo más a detalle a continuación:

- changeData(month, year): Permite la movilidad entre distintos meses o realizar el filtrado por año.
- renderEventsFromLocalStorage(): Se encarga de limpiar y cargar los eventos dentro del calendario, con la opción de visualizar los detalles.
- filtrarPorEstructura(): Se encarga de filtrar los resultados en tres diferentes formas (Todos, Pedidos o Suscripciones).
- abrirModalPedido(): Muestra los detalles de cualquier pedido que selecciones.
- agregarTarjetaSider(): Crea una tarjeta en el apartado de Próximos Eventos y se les asigna un comportamiento de acuerdo al tipo de evento(pedido o suscripción).

Blade

index.blade.php

Este mismo contiene la interfaz del Calendario, aunque no contiene varios elementos su desarrollo se compone de diferentes secciones.

Componentes de la Interfaz

La pantalla se conforma de tres distintas secciones:

1. **Calendario:** Muestra un calendario interactivo, donde se asignan los cards de acuerdo al día que corresponda y con la opción de seleccionarlo para visualizar sus detalles.
2. **Parámetros:** Encima del calendario y dentro del mismo se encuentran campos interactivos, que permiten cambiar el mes o seleccionar un año distinto.
3. **Próximos Eventos:** Un listado de los registros organizado por la fecha más cercana y con la posibilidad de filtrar por suscripciones, pedidos o mostrar todas en conjunto.

Modals

El apartado de Calendario se compone de simplemente dos diferentes modals, enfocados en dar detalles de acuerdo al elemento que se seleccione:

- **openDetail:** Muestra una ventana emergente con los detalles del pedido seleccionado.
- **openSub:** Muestra una ventana emergente con los detalles de la suscripción que se selecciona.

Llamado de Script

```
<script>
  const CALENDAR_URL = "{{ route('calendar.index') }}";

  window.CalendarData = {
    orders:      @json($orders),
    subscriptions: @json($subscriptions),
  };

  console.log('CalendarData:', window.CalendarData);
</script>
@push('scripts')
  <script src="{{ asset('js/calendar-logic.js') }}"?v={{ time() }}"></script>
@endpush
```

Al igual que en el caso anterior la llamada del archivo se compone de dos script:

- El primero define una URL para el controlador y crea un objeto global llamado **window.calendarData**, siendo una variable global que interactúa con el archivo js para obtener los pedidos y suscripciones registradas en el sistema..
- El segundo de ellos menciona la ruta final en donde puede encontrarse el archivo js, adicional a ello se usa `?v={{ time() }}`, esto nos funciona para que el navegador siempre cargue una versión reciente del archivo y no cargue versiones anteriores del caché.

Rutas

Para el apartado del Calendario únicamente requerimos la ruta de acceso al apartado para ser funcional:

```
Route::get('/newcalendar', [CalendarController::class, 'index'])->name('calendar.index');
```

Facturación

Este módulo proporciona una herramienta para realizar facturas a cualquier cliente que lo requiera, se solicitan los datos fiscales del cliente, estos son enviados a Facturapi para procesar la solicitud.

Controller

FacturaController.php

Se encarga de las peticiones HTTP del usuario y para devolver una respuesta, la información de las ventas se toma del modelo (sales) y se empaqueta en conjunto con lo que se ingresa del cliente. Las funciones más importantes son las siguientes:

- **create():** Obtiene las ventas del sistema y las organiza dentro de una tabla en un orden de 10 elementos visibles por cada página.

- **Facturar():** Recopila toda la información del cliente y la venta seleccionada para empaquetarla y enviarla a Facturapi para procesar la solicitud.
- **descargarArchivo():** Genera la ruta directa para descargar los archivos generados de la factura sin necesidad de entrar dentro de Facturapi.

Services

FacturacionService.php

Se encarga de la lógica del negocio que se relaciona con Facturapi, esté conecta ambos sistemas para generar la factura, devolviendo una respuesta que el controlador procesa para el punto de venta.

Lógica JS

factura-cliente.js

Las funciones se encargan de manejar las peticiones que el usuario genera, permitiendo seleccionar ventas, preparar los datos del formulario o realizar la búsqueda de clientes existentes en el sistema, modifica algunos aspectos de lo que se visualiza.

- **cargarVentas():** Recopila las ventas del sistema y las organiza en un listado para ser seleccionables.
- **seleccionarVenta():** Guarda la venta elegida y adapta su estructura para el formulario de facturación.
- **buscarCliente():** Consulta dentro del sistema la existencia del cliente que se escribe dentro de la barra de búsqueda.
- **seleccionarVenta():** Llena automáticamente el formulario fiscal con los datos del cliente seleccionado.

Blade

crear.blade.php

Contiene la interfaz con la que el usuario interactúa, mostrando el historial de ventas en su lado izquierdo y al lado contrario un formulario de datos fiscales que permite buscar clientes existentes o recopilar una nueva información que será enviada a Facturapi.

Ruta

Para el apartado de Facturación requerimos de diferentes rutas para crear una factura u obtener los archivos de Facturapi:

```
Route::get('/facturacion', function () { return view('pages.facturacion', ['title' => 'Facturación SAT']); }->name('facturacion');
Route::get('/factura/crear', [FacturaController::class, 'create']->name('factura.crear');
Route::post('/factura/crear', [FacturaController::class, 'facturar']->name('venta.facturar');
Route::get('/factura/archivo/{id}/{tipo?}', [FacturaController::class, 'descargarArchivo']->name('factura.archivo');
```